

Föreläsning 260515 – Samlingar – IEnumerable<T>

IEnumerable<T>

Är ett grundläggande gränssnitt i .NET som möjliggör **iteration** över en sekvens av objekt av typen T, och det är nyckeln till foreach-loopar och LINQ-frågor.

Vad är IEnumerable<T> ?

Tänk på IEnumerable<T> som en "kontrakt" som säger: "Den här samlingen kan du loopa igenom en i taget".

Varför använda det?

Vi använder IEnumerable<T> för att ge **minimalistisk (minimal) åtkomst** till data – **bara iteration, inget indexering eller räkning**.

Vi gör ingenting med elementen utan bara loopar igenom, t.ex. vid utskrift m.m.

Det främjar **abstraktion** så att din kod fungerar med alla samlingar (List, Array, Dictionary) utan att bry sig om implementationen.

Det aktiverar också LINQ-metoder som Where, Select och deferred execution, där koden körs först vid iteration. Vi återkommer till detta senare.

När ska vi använda det?

Returtyp i metoder: Returnera IEnumerable<T> för flexibilitet, t.ex. i API:er.

Parametrar: Ta emot IEnumerable<T> för att acceptera olika samlingstyper.

LINQ och strömmar: För stora dataset eller streaming, då det inte laddar allt i minnet på en gång.

Använd `IEnumerable<T>` istället för `List<T>` när man vill hålla koden **flexibel, minnesvänlig** och **abstrakt** – det är en lämplig returtyp eller parameter för **read-only** scenarier.

När ska vi använda `IEnumerable<T>`?

för återigen **read-only åtkomst** och **deferred execution** (koden körs inte förrän man itererar).

Det passar väldigt bra när man:

Returnerar data från metoder (t.ex. repository eller LINQ-frågor) utan att exponera implementationen.

Hanterar stora dataset eller streaming (t.ex. från DB), då det inte laddar allt i minnet direkt.

När ska vi använda `List<T>`?

När vi behöver **modifiera, indexera** eller använda **Count**:

Lägga till/ta bort element (Add, Remove).

Snabb åtkomst via index (`list[0]`).

Hur kan vi använda IEnumerable i ett garage?

`IEnumerable<T>` är ett steg mot bättre design i ett garage-system, där man t.ex. hanterar bilar utan att ladda allt i minnet på en gång.

I vår garage-app (t.ex. om vi har inventering av bilar eller reparationer) ger `IEnumerable<T>` **lazy loading** och **abstraktion**:
Vi loopar eller filtrerar utan att använda en full Lista.

Skulle kunna användas för stora listor från databas eller filer, som "hämta reparationer från idag".

Mer om hur IEnumerable <T> fungerar

Vad innebär denna kod?

```
public class Garage<T> : IEnumerable<T> where T : Vehicle
```

Här definierar vi en **generisk** klass **Garage<T>** som bara får innehålla typer som ärver från **Vehicle**, och att klassen kan användas i **foreach** eftersom den implementerar **IEnumerable<T>**.

Garage<T>: T är en typ-parameter, alltså en plats för en typ som bestäms när klassen används.

: IEnumerable<T>: klassen kan itereras, till exempel med `foreach`.

where T : Vehicle: Vi får bara använda typer som är en `Vehicle` eller ärver från `Vehicle`.

Vad det innebär i praktiken?

Det här är typiskt om vi vill ha ett garage som kan vara specialiserat, till exempel:

```
Garage<Car> carGarage = new Garage<Car>();
```

```
Garage<Motorcycle> bikeGarage = new Garage<Motorcycle>();
```

Då vet kompilatorn att varje objekt i garaget är en Vehicle-variant, vilket ger bättre typsäkerhet.

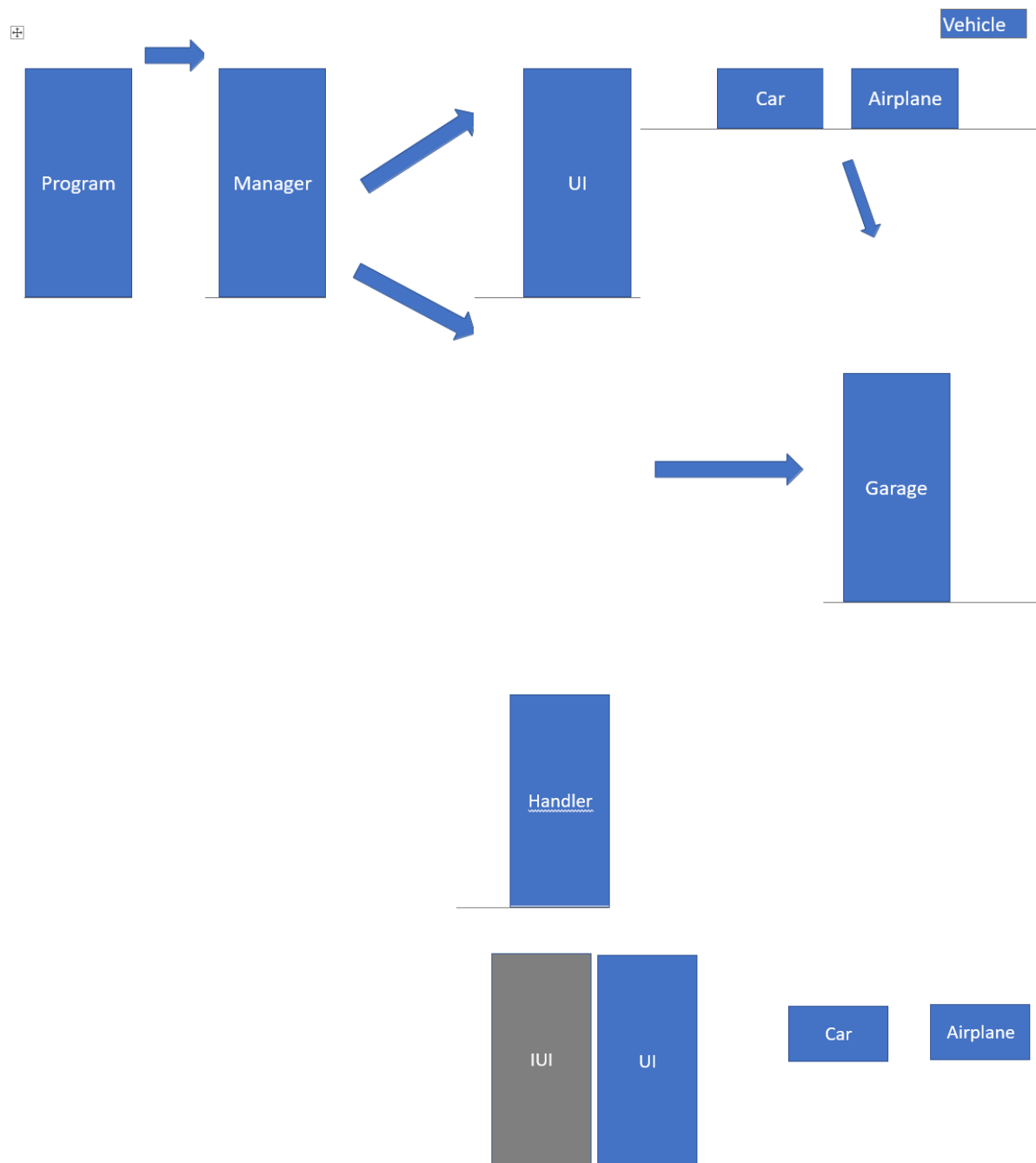
Ni kommer att använda er att denna kod:

```
public IEnumerable<T> GetEnumerator() =>  
    _vehicles.OfType<T>().GetEnumerator();
```

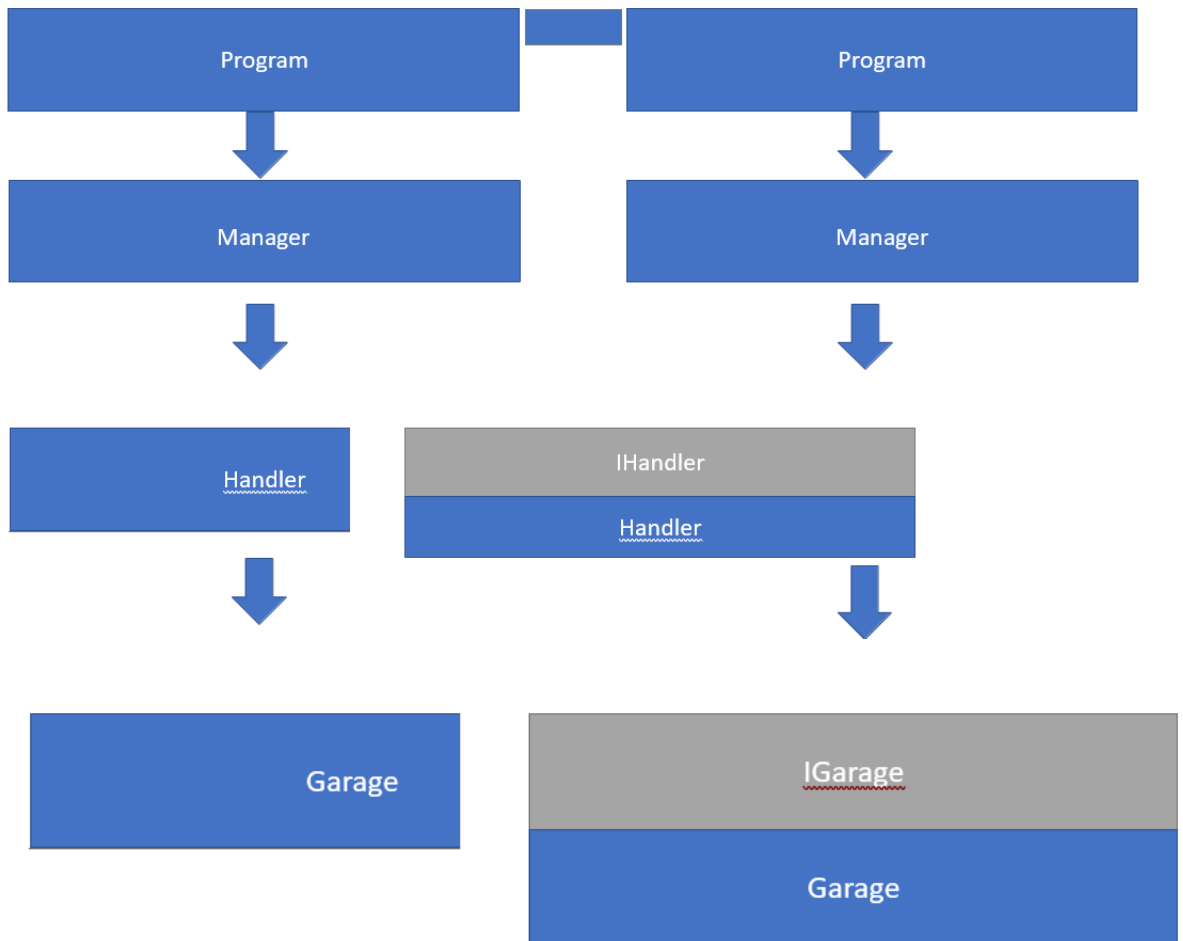
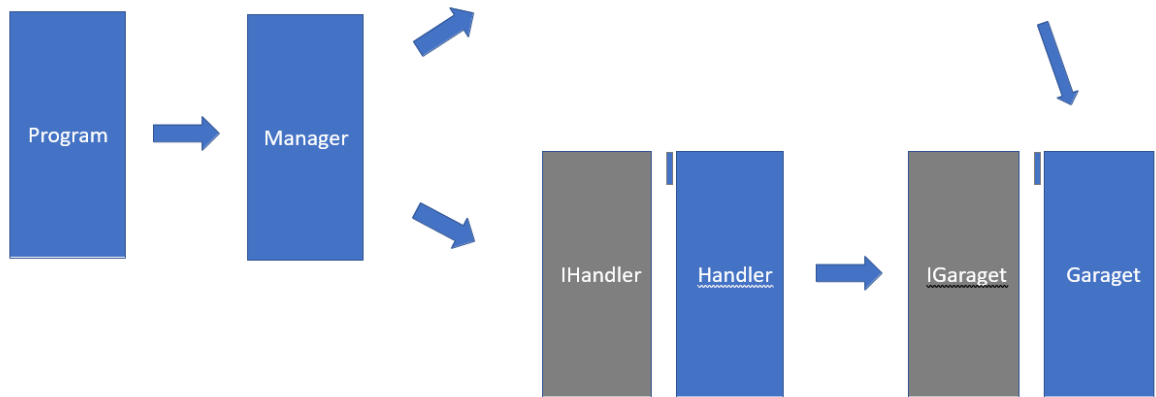
Den tar _vehicles, filtrerar fram bara de objekt som faktiskt är av typen T med OfType<T>(), och returnerar sedan en enumerator över dem.

Detta betyder att om _vehicles innehåller flera olika fordons-typer så får vi bara de som matchar T när vi loopar igenom samlingen.

Garage struktur:



Vehicle eller IVehicle



Med detta åstadkommer vi **loose coupling**: Vi programmerar mot gränssnitt i stället för mot konkreta klasser.

Det gör koden lättare att byta ut, testa och vidareutveckla, eftersom Manager inte behöver bry sig om exakt hur UI eller Garaget är byggda internt.